ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

ENTREGA FINAL CIERRE DE PROYECTO INICIAL “SLOTMACHINE”



JULIÁN DAVID GÓMEZ BOADA

DOCENTE

ANGIE TATIANA MEDINA GIL

MARIA IRMA DIAZ ROZO

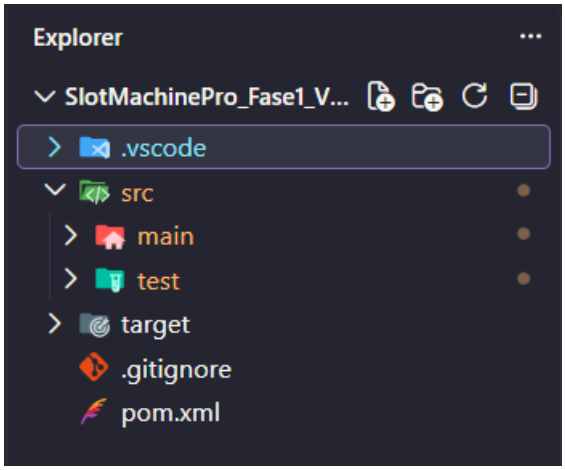
DISEÑO ORIENTADO POR OBJETOS

BOGOTÁ, CUNDINAMARCA

24 DE OCTUBRE DE 2026

Fase 1 — Preparación y migración a VS Code

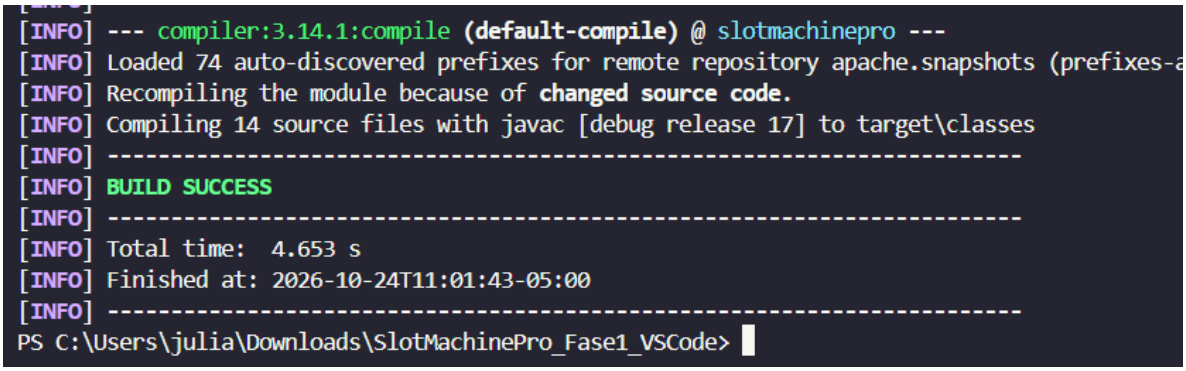
En esta primera fase trasladé la versión que ya tenía funcionando en BlueJ a Visual Studio Code, como lo pedía el instructivo. Organicé las 14 clases del programa y los ocho archivos de pruebas en una estructura de proyecto Java, procurando no cambiar nada de su funcionamiento.



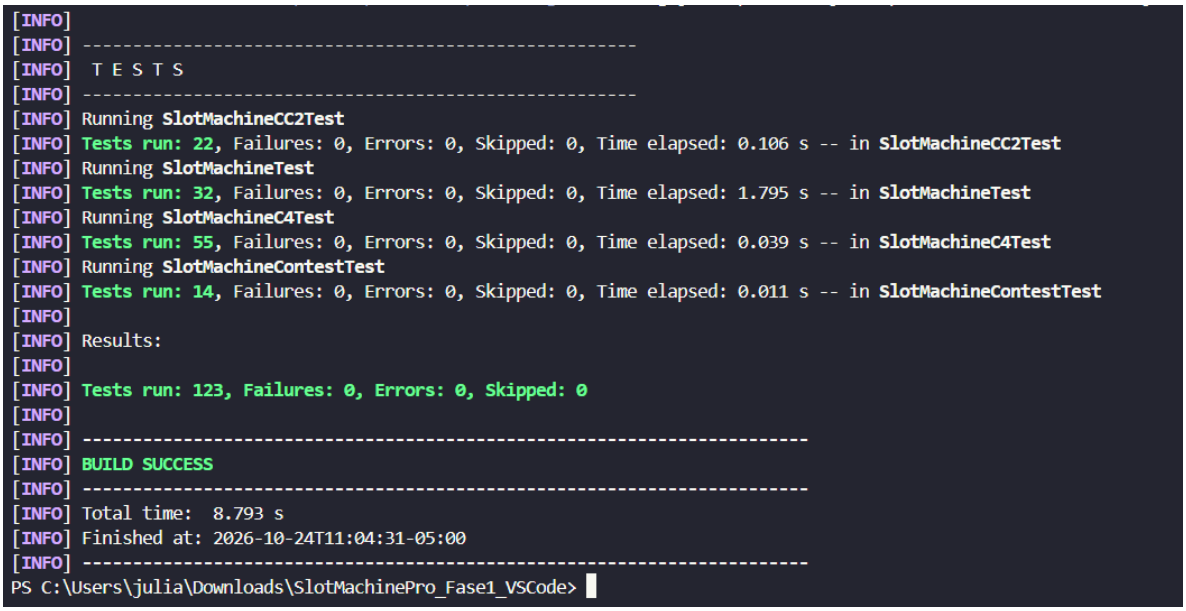
Utilicé las carpetas src/main/java para el código del programa y src/test/java para las pruebas. También preparé un archivo pom.xml de Maven, que me permitió manejar las librerías necesarias desde un mismo lugar. Noté que mis pruebas usaban JUnit 5 en los ciclos 1 y 2 y JUnit 4 en los ciclos 3 y 4; por eso configuré JUnit Jupiter y JUnit Vintage, en vez de cambiar pruebas que ya había desarrollado.

```
PS C:\Users\julia\Downloads\SlotMachinePro_Fase1_VSCode> mvn clean compile
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< edu.eci.dopo:slotmachinepro >--------------------
[INFO] Building SlotMachinePro - Cierre Proyecto Inicial 1.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO] Loaded 24227 auto-discovered prefixes for remote repository central (prefixes-central.txt)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plu
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-clean-plug
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/4
```

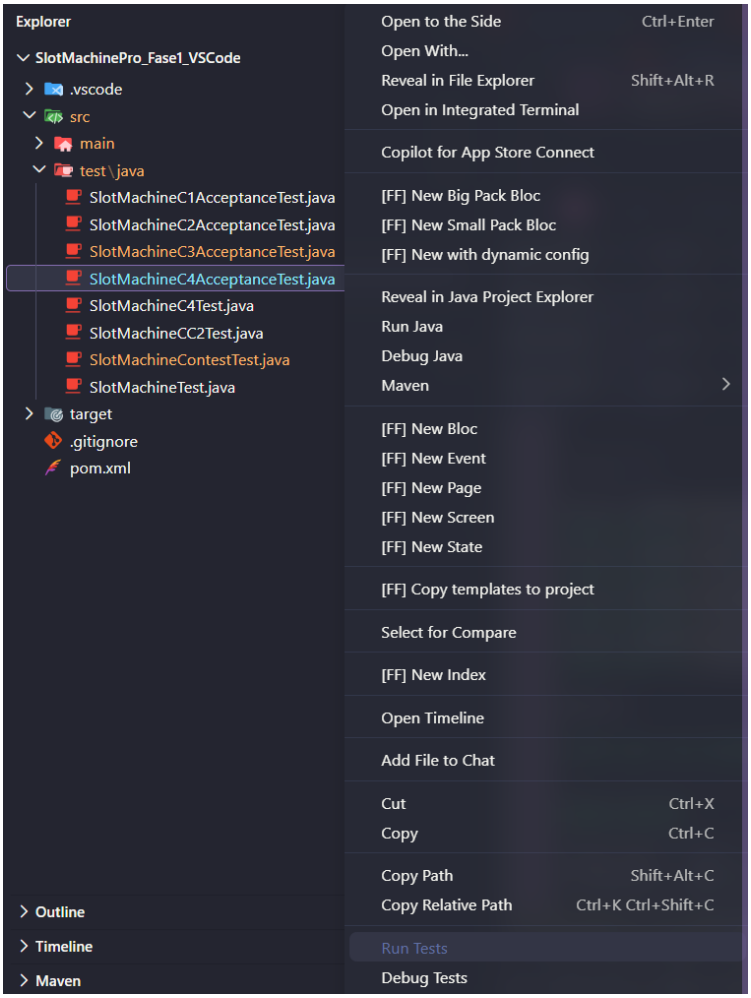
Con la estructura lista, ejecuté mvn clean compile desde la terminal de VS Code. El resultado fue BUILD SUCCESS y se compilaron correctamente las 14 clases del proyecto. Consideré importante comprobar esto primero, porque antes de estudiar la calidad del código necesitaba asegurarme de que la migración no había producido errores.



Después ejecuté mvn test para revisar las pruebas unitarias originales. Maven informó 123 pruebas ejecutadas, sin fallos, errores ni pruebas omitidas. Separé las pruebas de aceptación de este comando para poder ejecutarlas de forma controlada cuando fuera necesario, sobre todo en los casos que podían mostrar ventanas gráficas.



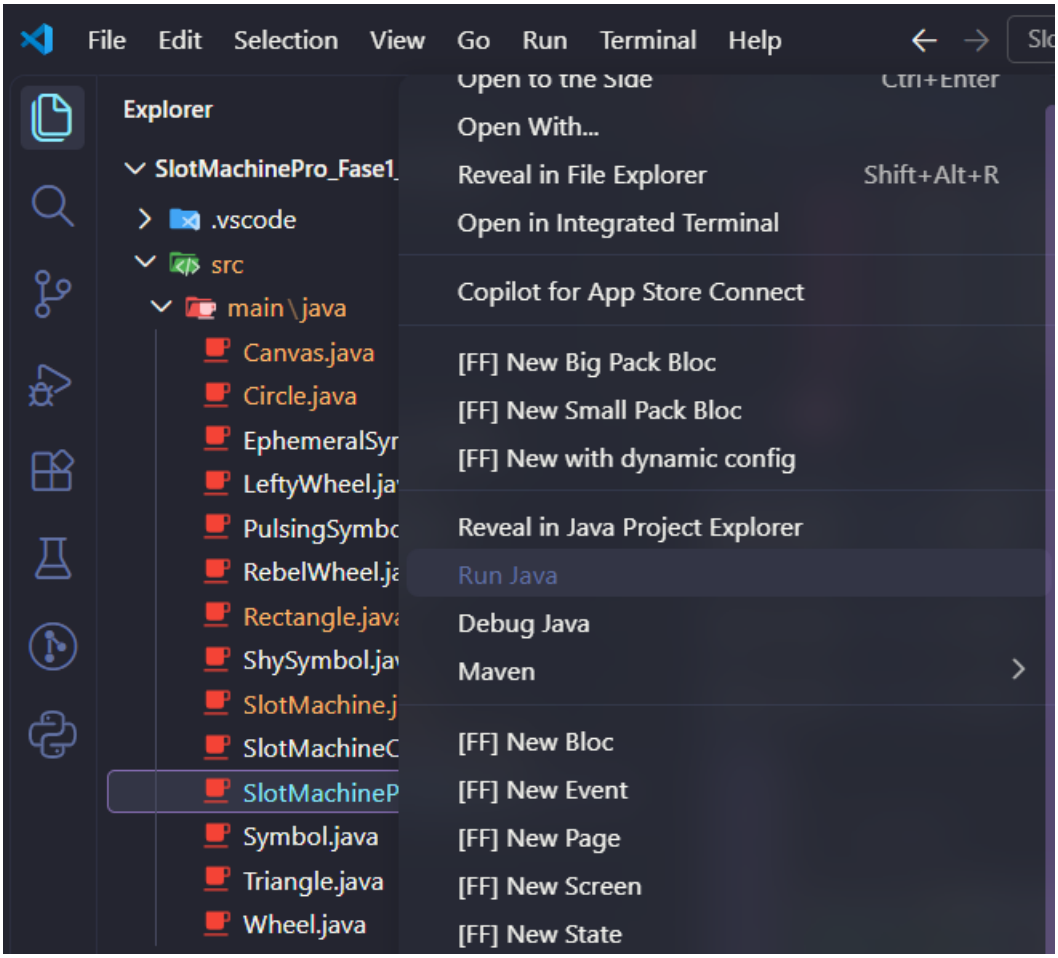
También revisé la ejecución de las pruebas desde el panel de Java de VS Code. Para la evidencia seleccioné las pruebas de aceptación del Ciclo 4, ya que reúnen ruedas y símbolos especiales, restricciones y la evaluación del jackpot.



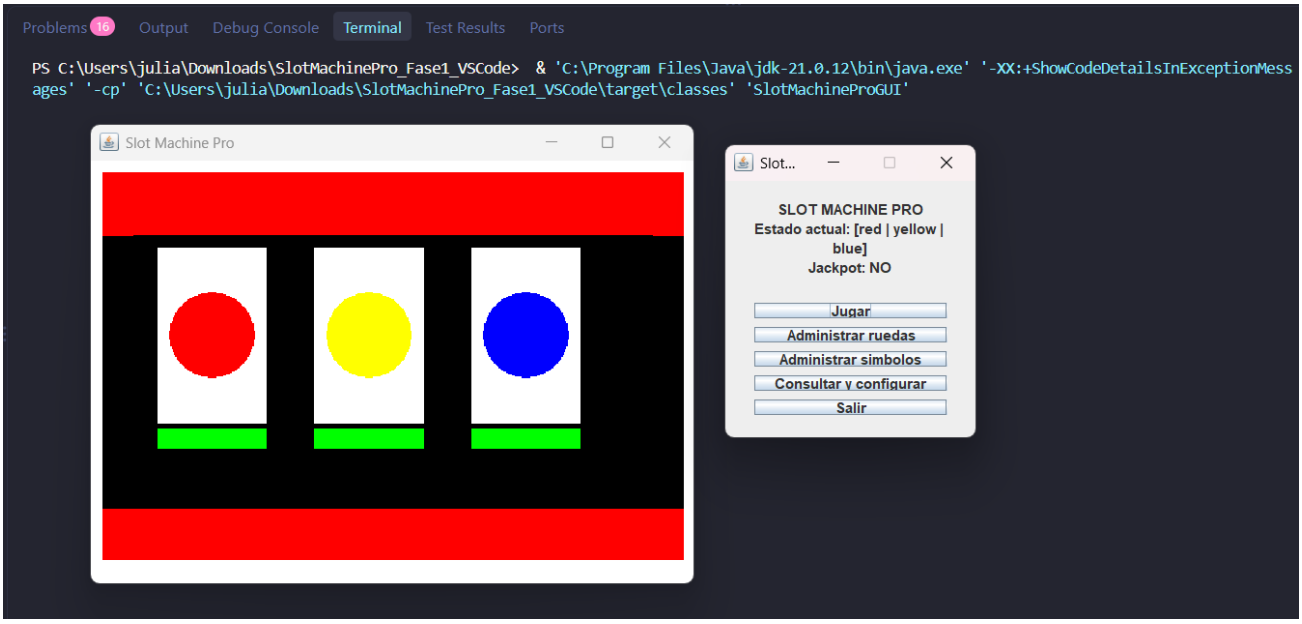
Las dos pruebas de aceptación del Ciclo 4 aparecieron aprobadas en el Test Runner for Java. Esto me permitió comprobar que, además de compilar, podía ejecutar escenarios completos del proyecto desde el nuevo entorno.



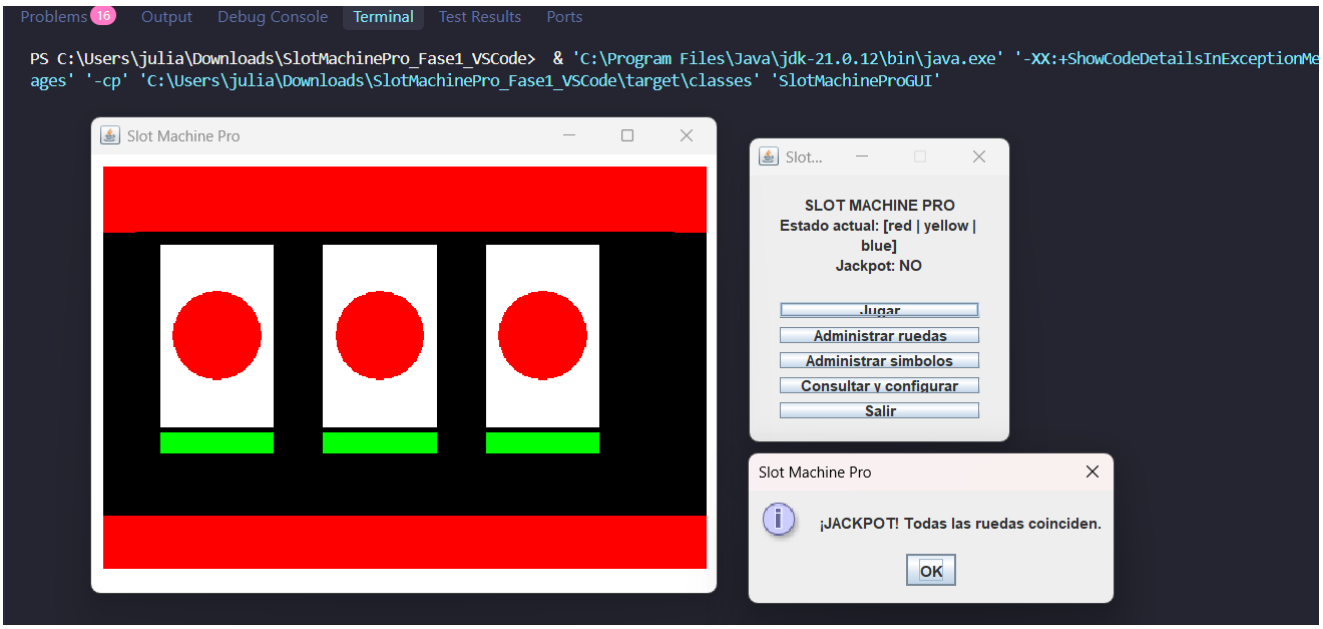
Por último, abrí la clase SlotMachineProGUI y ejecuté su método main. Quise confirmar de manera visual que la ventana de opciones y el dibujo de la máquina seguían funcionando después del traslado.



La interfaz abrió correctamente con la configuración inicial [red | yellow | blue]. Conservó sus botones para jugar, administrar ruedas y símbolos, y consultar o configurar el estado de la máquina.

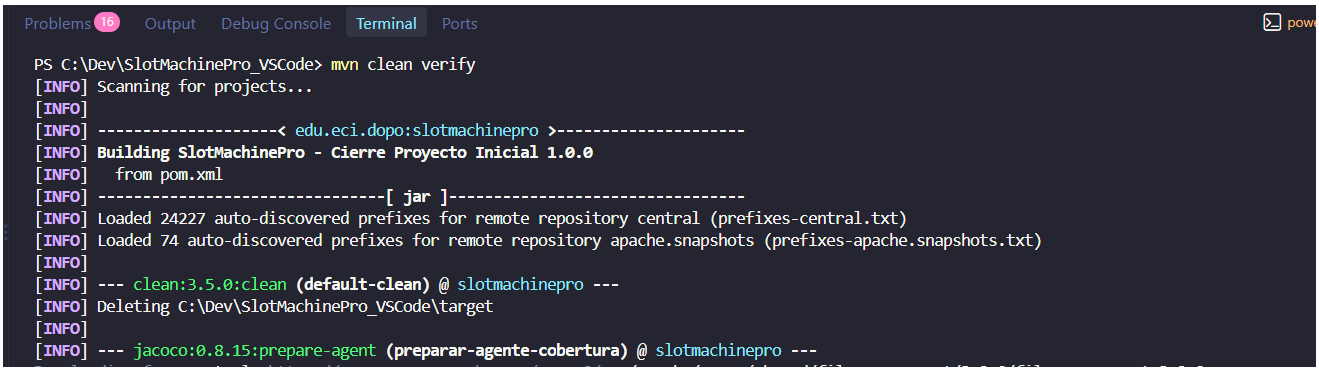


Finalmente, realicé tres giros generales seguidos y obtuve la Jackpot con su respectivo aviso. Con esta revisión consideré terminada la migración: el proyecto compilaba, las pruebas originales pasaban y la representación visual continuaba funcionando.

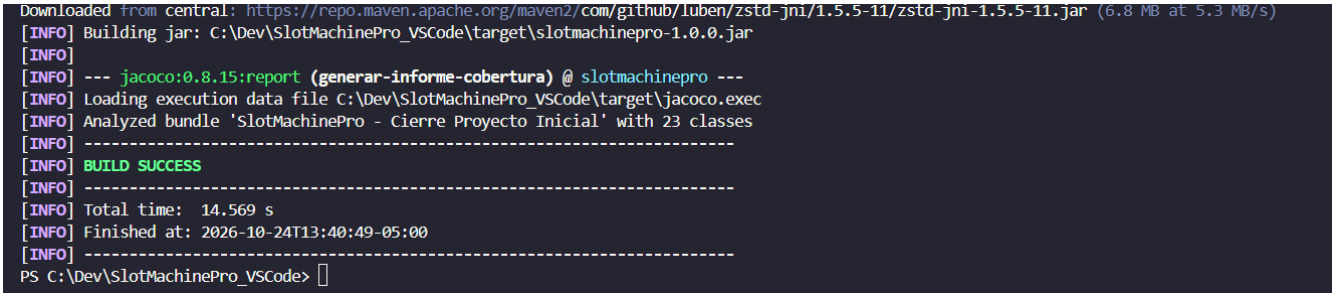


Fase 2 — Análisis dinámico inicial con JaCoCo

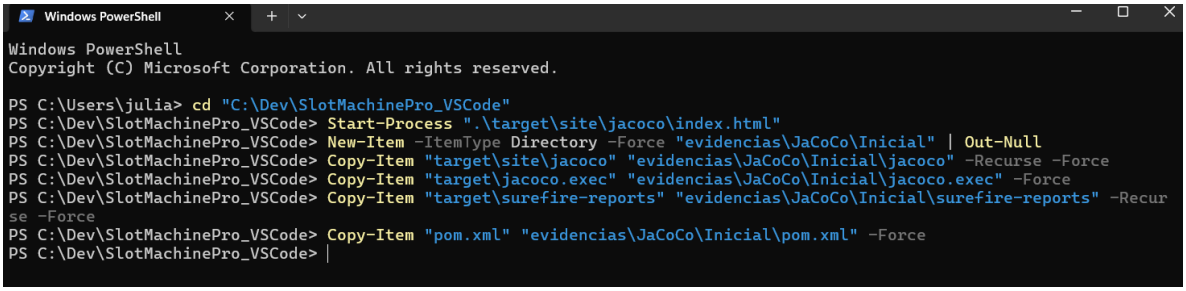
En esta fase quise conocer qué tanto código estaba ejecutando realmente mis pruebas. Agregué JaCoCo al pom.xml y ejecuté mvn clean verify, que permitió correr las pruebas y crear un informe de cobertura sin cambiar las clases del proyecto.



Cuando terminó Maven, obtuve BUILD SUCCESS y se generó el informe en target/site/jacoco/index.html. A diferencia de las pruebas aprobadas, este resultado me permitía observar cuáles instrucciones, líneas y decisiones del programa sí habían sido recorridas.

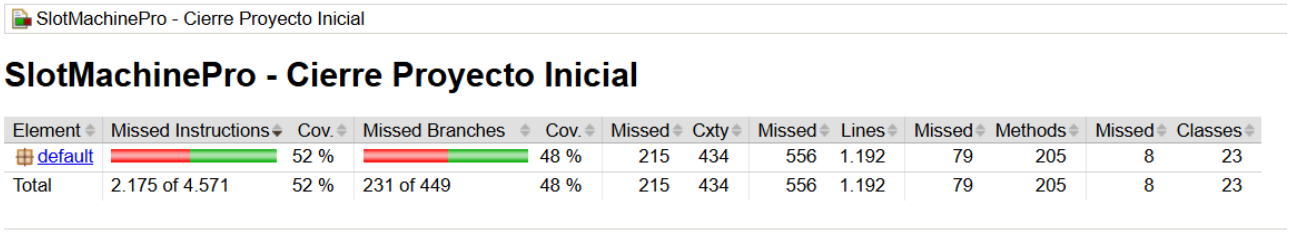


Antes de hacer cualquier corrección, guardé una copia del informe HTML, el archivo jacoco.exec, los resultados de Surefire y el pom.xml utilizado. Tomé esta decisión porque el instructivo pide comparar un resultado inicial con uno final, y necesitaba conservar una evidencia que no se sobrescribiera al repetir el análisis.



Resultados iniciales y análisis

La primera medición mostró 52,42 % de instrucciones cubiertas, 53,36 % de líneas y 48,55 % de ramas. Noté que los valores estaban por debajo de la meta de más del 75 % solicitada por la asignatura, por lo que decidí revisar el detalle por clases antes de agregar pruebas sin una razón clara.



Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
default		52 %		48 %	215	434	556	1.192	79	205	8	23
Total	2.175 of 4.571	52 %	231 of 449	48 %	215	434	556	1.192	79	205	8	23

Al entrar al desglose encontré que SlotMachine tenía 96 % de cobertura de instrucciones y Wheel 87 %. Las clases EphemeralSymbol, ShySymbol y PulsingSymbol llegaban al 100 %. Esto me hizo pensar que el problema principal no estaba en las reglas de la máquina ni en las especializaciones que había trabajado en el Ciclo 4.

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
SlotMachineProGUI		0 %		0 %	118	118	396	396	36	36	1	1
Triangle		34 %		16 %	13	22	34	66	7	16	0	1
Wheel		87 %		81 %	13	60	20	171	2	27	0	1
Rectangle		61 %		33 %	10	22	24	64	6	16	0	1
Circle		60 %		33 %	10	21	24	61	6	15	0	1
SlotMachineContest		46 %		35 %	6	11	14	28	1	4	0	1
SlotMachine		96 %		86 %	19	98	16	254	2	30	0	1
Canvas		87 %		69 %	5	22	7	68	0	9	0	1
Symbol		44 %		0 %	4	12	8	18	3	11	0	1
SlotMachineProGUI.new Runnable() {...}		0 %		n/a	2	2	4	4	2	2	1	1
SlotMachineProGUI.new ActionListener() {...}		0 %		n/a	2	2	2	2	2	2	1	1
SlotMachineProGUI.new ActionListener() {...}		0 %		n/a	2	2	2	2	2	2	1	1
SlotMachineProGUI.new ActionListener() {...}		0 %		n/a	2	2	2	2	2	2	1	1
SlotMachineProGUI.new ActionListener() {...}		0 %		n/a	2	2	2	2	2	2	1	1
SlotMachineProGUI.new ActionListener() {...}		0 %		n/a	2	2	2	2	2	2	1	1
SlotMachineProGUI.new WindowAdapter() {...}		0 %		n/a	2	2	2	2	2	2	1	1
LeftyWheel		85 %		83 %	2	6	2	11	1	3	0	1
RebelWheel		83 %		n/a	1	5	1	6	1	5	0	1
EphemeralSymbol		100 %		100 %	0	6	0	10	0	5	0	1
PulsingSymbol		100 %		100 %	0	7	0	10	0	5	0	1
Canvas.ShapeDescription		100 %		n/a	0	2	0	8	0	2	0	1
ShySymbol		100 %		100 %	0	6	0	8	0	5	0	1
Canvas.CanvasPane		100 %		n/a	0	2	0	3	0	2	0	1
Total	2.175 of 4.571	52 %	231 of 449	48 %	215	434	556	1.192	79	205	8	23

En cambio, observé que SlotMachineProGUI tenía 0 % de cobertura y concentraba 396 líneas sin ejecutar. También aparecían con cobertura baja Triangle (34 %), Symbol (44 %), SlotMachineContest (46 %), Circle (60 %) y Rectangle (61 %). Analicé que mis pruebas trabajaban sobre todo directamente con SlotMachine y casi nunca activaban los botones y métodos de la interfaz.

Por eso decidí que, cuando llegara el momento de mejorar el código, daría prioridad a probar la GUI y completaría algunos casos de las clases con menor cobertura. Preferí no excluir la interfaz del informe para subir el porcentaje de manera artificial ni modificar clases que ya tenían buena cobertura. Dejé estas decisiones registradas para la fase de refactorización.

Fase 3 — Análisis estático inicial con PMD

En la tercera fase revisé la calidad del código sin ejecutar el simulador. Agregué PMD al pom.xml y usé mvn test-compile pmd:pmd. Mantuve la misma versión del proyecto con la que había tomado la primera medición de JaCoCo, para saber exactamente qué observaciones existían antes de los cambios.

```
[INFO] Rendering content with org.apache.maven.skins:maven-fluido-skin:jar:2.0.0-M9 skin
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  17.366 s
[INFO] Finished at: 2026-10-24T14:58:21-05:00
[INFO] ------------------------------------------------------------------------
PS C:\Dev\SlotMachinePro_VSCode>
```

El análisis terminó correctamente y pude abrir el informe HTML. Además, ejecuté un pequeño resumen en PowerShell para contar las advertencias por prioridad. PMD encontró cuatro en total: dos de prioridad 3 y dos de prioridad 4, sin problemas de prioridad 1 o 2.

```
PS C:\Dev\SlotMachinePro_VSCode> Start-Process ".\target\reports\pmd.html"
PS C:\Dev\SlotMachinePro_VSCode> powershell -NoProfile -ExecutionPolicy Bypass -File ".\resumen_pmd.ps1"

SLOTMACHINEPRO - RESUMEN PMD REAL
Total de incidencias: 4
  Prioridad 1: 0
  Prioridad 2: 0
  Prioridad 3: 2
  Prioridad 4: 2
  Prioridad 5: 0
Pendientes prioridad 1 y 2: 0

Reglas mas frecuentes:

Count Name
----- ----
    1 UnnecessaryFullyQualifiedName
    1 UselessParentheses
    1 EmptyCatchBlock
    1 UnusedPrivateField



Incidencias de prioridades 1 y 2:

Detalle CSV: .\target\diagnostico-pmd\incidencias-pmd.csv
PS C:\Dev\SlotMachinePro_VSCode>
```

Análisis de las observaciones y decisiones

Al revisar los resultados noté que ya cumplía la meta obligatoria de no tener problemas de prioridad alta. Aun así, consideré conveniente corregir las cuatro observaciones, porque eran cambios pequeños y me permitirían mostrar una mejora clara entre los informes inicial y final.

SlotMachinePro - Cierre Proyecto Inicial

Last Published: 1980-02-01 | Version: 1.0.0 SlotMachinePro - Cierre Proyecto Inicial

PMD Results

The following document contains the results of PMD 7.17.0.

Violations By Priority

Priority 3

Canvas.java

Rule	Violation	Line
EmptyCatchBlock	Avoid empty catch blocks	148–150

SlotMachine.java

Rule	Violation	Line
UnusedPrivateField	Avoid unused private fields such as 'visible'.	17

Las dos observaciones de prioridad 3 estaban en Canvas.java y SlotMachine.java. En Canvas, PMD encontró un bloque catch que no atendía la interrupción de Thread.sleep(); decidí manejar InterruptedException y conservar la señal con Thread.currentThread().interrupt(). En SlotMachine se había declarado el atributo privado visible, pero no se utilizaba para consultar el estado; por eso decidí quitarlo junto con sus asignaciones, sin tocar los métodos que muestran y ocultan las figuras.

Priority 4

SlotMachineProGUI.java

Rule	Violation	Line
UnnecessaryFullyQualifiedName	Unnecessary qualifier 'JFrame': 'WindowConstants::DO_NOTHING_ON_CLOSE' is already in scope because it is inherited by an enclosing type	94

Wheel.java

Rule	Violation	Line
UselessParentheses	Useless parentheses around `sx + 75`.	71

Las dos observaciones de prioridad 4 eran de escritura. En SlotMachineProGUI podía usar DO_NOTHING_ON_CLOSE sin repetir JFrame, porque la clase ya hereda de JFrame. En Wheel había unos paréntesis innecesarios en el cálculo de la posición de un marcador. Decidí simplificar ambas instrucciones, ya que el resultado y el comportamiento seguirían siendo los mismos.

Files

Canvas.java

Rule	Violation	Priority	Line
EmptyCatchBlock 🎈	Avoid empty catch blocks	3	148–150

SlotMachine.java

Rule	Violation	Priority	Line
UnusedPrivateField 🎈	Avoid unused private fields such as 'visible'.	3	17

SlotMachineProGUI.java

Rule	Violation	Priority	Line
UnnecessaryFullyQualifiedName 🎈	Unnecessary qualifier 'JFrame': 'WindowConstants::DO_NOTHING_ON_CLOSE' is already in scope because it is inherited by an enclosing type	4	94

Wheel.java

Rule	Violation	Priority	Line
UselessParentheses 🎈	Useless parentheses around `sx + 75`.	4	71

En conjunto, mi criterio fue corregir lo señalado por PMD con el menor cambio posible. No consideré necesario rediseñar las clases, cambiar operaciones públicas ni alterar la forma prepareView()–prepareController() de la GUI que tanto me costó construir. Guardé el informe XML, el HTML y el resumen inicial antes de pasar a las correcciones.

```
Detalle CSV: .\target\diagnostico-pmd\incidencias-pmd.csv
PS C:\Dev\SlotMachinePro_VSCode> New-Item -ItemType Directory -Force "evidencias\PMD\Inicial" | Out-Null
PS C:\Dev\SlotMachinePro_VSCode> Copy-Item "target\pmd.xml" "evidencias\PMD\Inicial\pmd.xml" -Force
PS C:\Dev\SlotMachinePro_VSCode> Copy-Item "target\reports\pmd.html" "evidencias\PMD\Inicial\pmd.html" -Force
PS C:\Dev\SlotMachinePro_VSCode> Copy-Item "pom.xml" "evidencias\PMD\Inicial\pom.xml" -Force
PS C:\Dev\SlotMachinePro_VSCode> Copy-Item "target\diagnostico-pmd\incidencias-pmd.csv" "evidencias\PMD\Inicial\incidenc
ias-pmd.csv" -Force
PS C:\Dev\SlotMachinePro_VSCode>
```

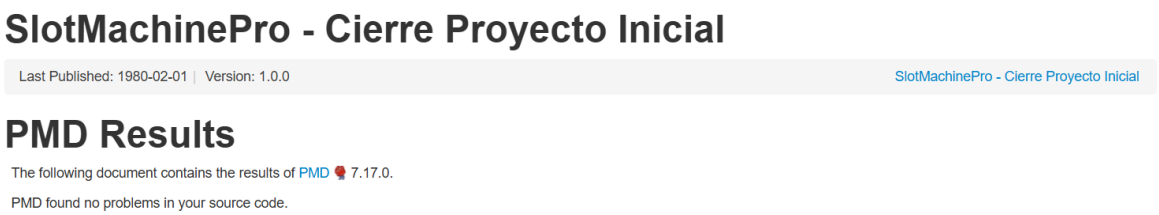
Fase 4 — Refactorización controlada y mediciones finales

Con los dos diagnósticos listos, empecé a aplicar las mejoras. Mi intención fue resolver primero las cuatro observaciones de PMD y después aumentar la cobertura de JaCoCo, pero siempre comprobando que el simulador siguiera haciendo lo mismo que antes.

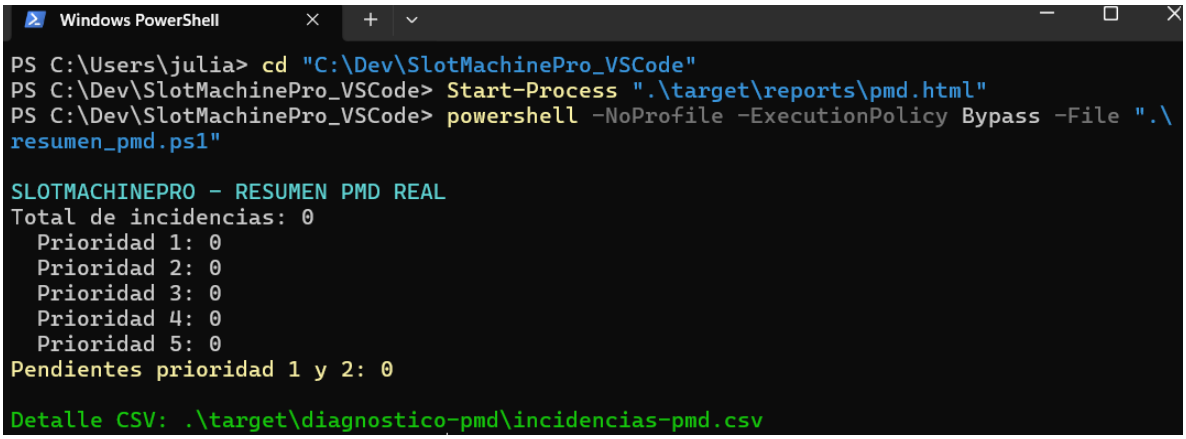
a) Correcciones de PMD

Realicé las modificaciones puntuales en Canvas, SlotMachine, SlotMachineProGUI y Wheel. Después compilé nuevamente el proyecto, ejecuté

las pruebas y repetí el análisis con la misma configuración de PMD. El informe dejó de mostrar observaciones.

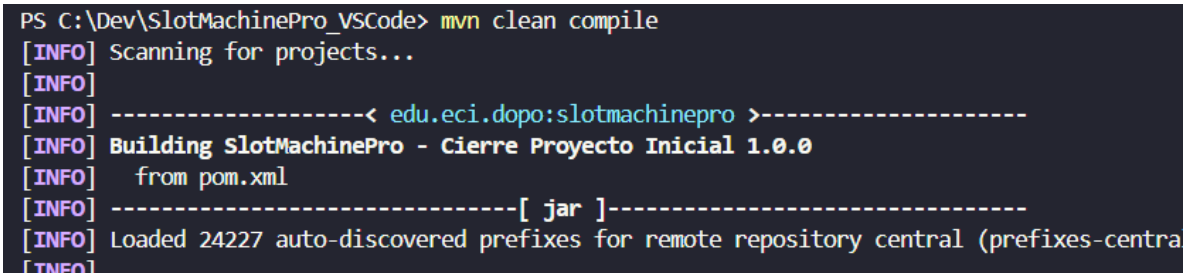


El resumen confirmó cero incidencias en las cinco prioridades. Pienso que este resultado es importante porque no fue necesario hacer una refactorización grande: bastó con atender los problemas concretos que había encontrado la herramienta. También conservé las versiones anteriores para poder explicar qué cambió.

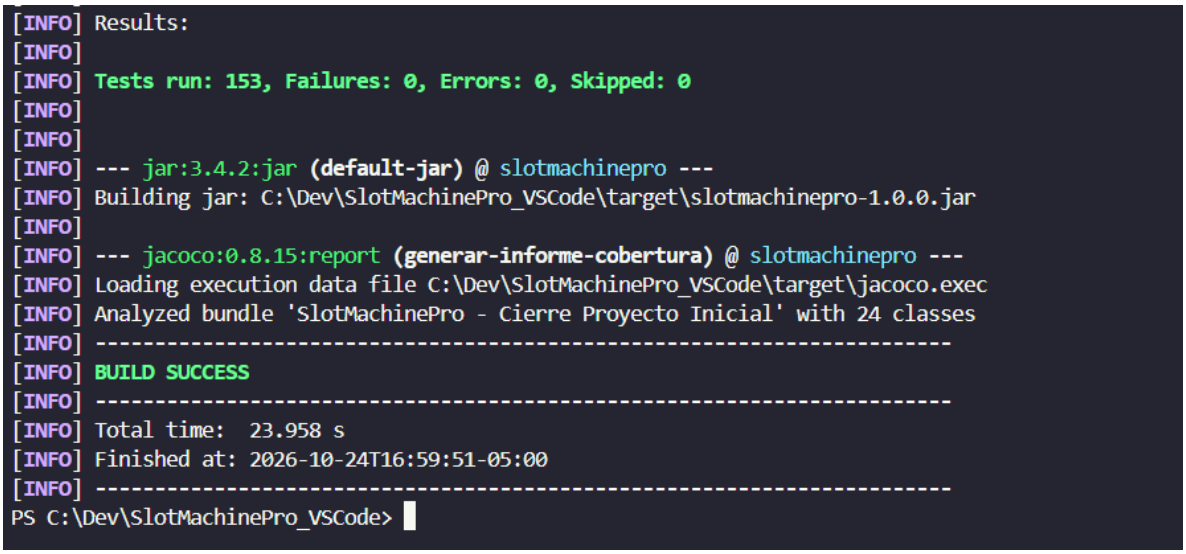


b) Ampliación de cobertura con JaCoCo

Para mejorar JaCoCo me enfoqué principalmente en SlotMachineProGUI, ya que antes tenía 0 % de cobertura. Noté que probar sus menús podía dejar una prueba esperando una respuesta en JOptionPane, así que incorporé DialogPort y SwingDialogPort dentro de la GUI. Durante el uso normal seguían apareciendo los mismos diálogos, pero en las pruebas podía simular las respuestas sin detener la ejecución.



Agregué dos archivos de pruebas: SlotMachineProGUICoverageTest.java, con 23 casos para menús, botones y operaciones de la interfaz, y SlotMachineCoverageComplementTest.java, con 7 casos para completar comportamientos de Symbol, SlotMachineContest y las figuras gráficas. Consideré mejor ampliar las pruebas que cambiar las reglas del dominio, porque estas ya habían mostrado una cobertura alta.



Resultados finales y comprobación

Después ejecuté otra vez mvn clean verify. Maven informó 153 pruebas unitarias aprobadas, sin fallos, errores ni omisiones, y generó la nueva medición de JaCoCo. Así pude comprobar que las 30 pruebas añadidas funcionaban junto a las 123 originales.

SlotMachinePro - Cierre Proyecto Inicial

SlotMachinePro - Cierre Proyecto Inicial

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
default		97 %		92 %	45	440	41	1.199	9	210	1	24
Total	132 of 4.657	97 %	36 of 451	92 %	45	440	41	1.199	9	210	1	24

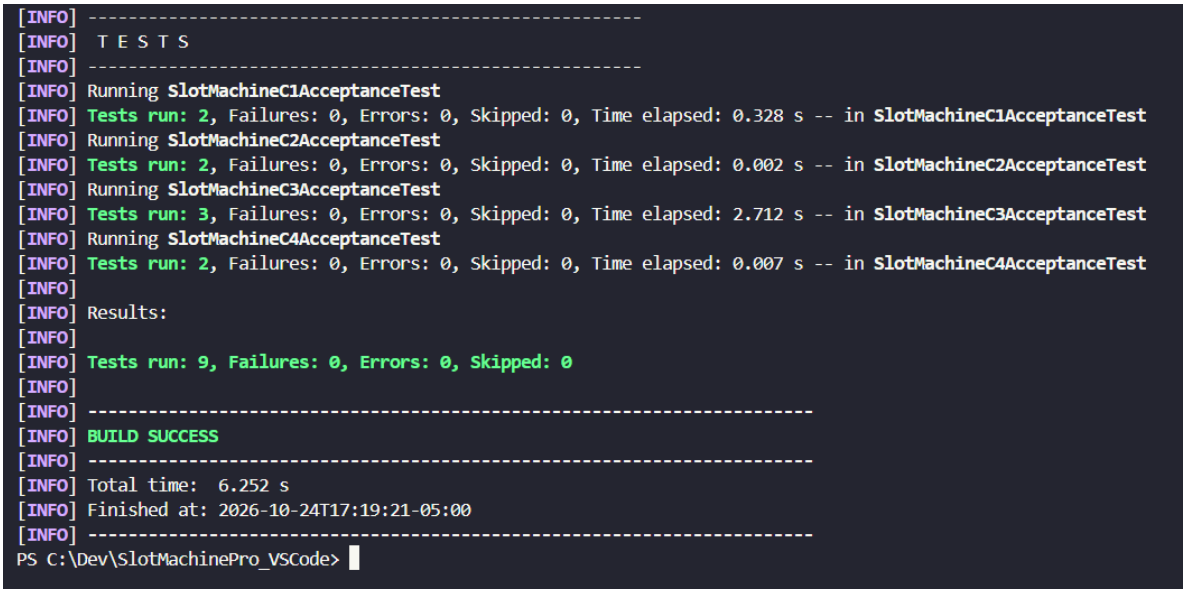
La cobertura de instrucciones subió de 52,42 % a 97,17 %; la de líneas pasó de 53,36 % a 96,58 %, y la de ramas de 48,55 % a 92,02 %. Con estos datos comprobé que superé la meta del 75 %. El total de instrucciones analizadas también cambió ligeramente por la adaptación realizada en la GUI, de modo que comparé tanto porcentajes como cantidades cubiertas.

SlotMachinePro - Cierre Proyecto Inicial > default

default

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Wheel		94 %		87 %	9	60	7	171	1	27	0	1
SlotMachineProGUI		98 %		92 %	13	120	12	401	1	37	0	1
SlotMachineProGUI.SwingDialogPort		0 %		n/a	4	4	5	5	4	4	1	1
SlotMachine		97 %		88 %	16	98	12	250	1	30	0	1
SlotMachineProGUI.new ActionListener() {...}		60 %		n/a	1	2	1	2	1	2	0	1
SlotMachineProGUI.new WindowAdapter() {...}		60 %		n/a	1	2	1	2	1	2	0	1
Canvas		99 %		100 %	0	22	2	69	0	9	0	1
LeftyWheel		92 %		83 %	1	6	1	11	0	3	0	1
Triangle		100 %		100 %	0	22	0	66	0	16	0	1
Rectangle		100 %		100 %	0	22	0	64	0	16	0	1
Circle		100 %		100 %	0	21	0	61	0	15	0	1
SlotMachineContest		100 %		100 %	0	11	0	28	0	4	0	1
Symbol		100 %		100 %	0	12	0	18	0	11	0	1
EphemeralSymbol		100 %		100 %	0	6	0	10	0	5	0	1
PulsingSymbol		100 %		100 %	0	7	0	10	0	5	0	1
Canvas.ShapeDescription		100 %		n/a	0	2	0	8	0	2	0	1
ShySymbol		100 %		100 %	0	6	0	8	0	5	0	1
Canvas.CanvasPane		100 %		n/a	0	2	0	3	0	2	0	1
RebelWheel		100 %		n/a	0	5	0	6	0	5	0	1
SlotMachineProGUI.new Runnable() {...}		100 %		n/a	0	2	0	4	0	2	0	1
SlotMachineProGUI.new ActionListener() {...}		100 %		n/a	0	2	0	2	0	2	0	1
SlotMachineProGUI.new ActionListener() {...}		100 %		n/a	0	2	0	2	0	2	0	1
SlotMachineProGUI.new ActionListener() {...}		100 %		n/a	0	2	0	2	0	2	0	1
SlotMachineProGUI.new ActionListener() {...}		100 %		n/a	0	2	0	2	0	2	0	1
Total	132 of 4.657	97 %	36 of 451	92 %	45	440	41	1.199	9	210	1	24

El desglose final mostró una mejora clara en las clases que había identificado: SlotMachineProGUI llegó al 98 %, SlotMachineContest y Symbol al 100 %, y Triangle, Circle y Rectangle también al 100 %. Una clase interna encargada de mostrar diálogos reales quedó sin cubrir, pero decidí no añadir cambios innecesarios solo para llegar al 100 % general. La nueva cobertura ya era suficiente y el objetivo principal era conservar un programa estable.



Para terminar la verificación ejecuté por separado las nueve pruebas de aceptación de los cuatro ciclos y todas pasaron. Esto complementó las 153 unitarias y me permitió comprobar que los cambios de calidad no afectaron los escenarios completos del proyecto. También revisé el funcionamiento visual de la máquina y de sus opciones principales.

Fase 5 — Preparación de entregables finales

Finalmente, revisé que el diseño de Astah estuviera actualizado y correspondiera con las clases entregadas, incluyendo las especializaciones y la organización de la interfaz gráfica. Organicé el proyecto de VS Code, las pruebas y las evidencias iniciales y finales de JaCoCo y PMD para que se pueda verificar el trabajo realizado en cada etapa.

En este informe dejé descritos el procedimiento, los resultados y las razones de los cambios que tomé. La retrospectiva del proyecto, con el estado de los miniciclos/fases y retrospectiva de ciclos anteriores de este cierre, los elaboré por separado y se incorporaron después a la entrega definitiva.